

Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №2
За седьмой семестр
По дисциплине: «Интеллектуальный анализ данных»
Тема: Автоэнкодеры и визуализация выборки WDBC

Выполнил:
Студент 4 курса
Группы ИИ-26(2)
Семёнов А.В.
Проверил:
Никитин С.К.

Брест 2026

Автоэнкодеры и визуализация выборки WDBC

1 Условие работы и подготовка данных

Цель работы — научиться применять автоэнкодеры для визуализации многомерных данных и сравнить их с t-SNE и методом главных компонент. По варианту 1 используется Wisconsin Diagnostic Breast Cancer (WDBC). Для каждого метода строятся двумерное и трёхмерное представления с различными цветами и формами маркеров для двух классов.

Выборка загружается функцией `sklearn.datasets.load_breast_cancer`. Она содержит 569 объектов и 30 числовых признаков. В исходном формате WDBC первый столбец — идентификатор, второй — диагноз М/В; они не входят в матрицу признаков. В версии `scikit-learn` идентификатор исключён, диагноз вынесен в `target`: 0 соответствует М, 1 соответствует В. Пропусков и бесконечных значений нет.

Класс	Обозначение	Число объектов	Доля, %
Злокачественные	М / 0	212	37,26
Доброкачественные	В / 1	357	62,74

Тридцать признаков образованы десятью характеристиками ядер клеток: радиусом, текстурой, периметром, площадью, гладкостью, компактностью, вогнутостью, числом вогнутых точек, симметрией и фрактальной размерностью. Для каждой характеристики приведены среднее, стандартная ошибка и значение `worst` — среднее трёх наибольших измерений.

Стандартизация и схема эксперимента

Признаки имеют разные масштабы, поэтому каждый столбец центрируется и делится на выборочное стандартное отклонение, как в первой лабораторной работе. Метки классов не участвуют в преобразовании, обучении или подборе параметров.

$$z_{ij} = (x_{ij} - \mu_j) / s_j; \quad s_j^2 = \sum_i (x_{ij} - \mu_j)^2 / (n - 1)$$

Для подбора автоэнкодера случайным разбиением с `seed = 42` выделены 341 обучающий объект, 114 валидационных и 114 тестовых. Средние и масштабы в этом эксперименте вычисляются только на обучающей части. Архитектура и число эпох выбираются по валидации; тест применяется после выбора модели.

Для итоговых графиков выбранный автоэнкодер повторно обучается на всех 569 объектах с числом эпох, найденным на валидации. PCA и t-SNE для визуализации также получают полную стандартизованную выборку. Результаты на всей выборке и тестовой части далее указаны отдельно.

2 Модель автоэнкодера

Автоэнкодер состоит из кодировщика, сжимающего 30 признаков до k координат, и декодировщика, восстанавливающего вход. Использованы обычные полносвязные слои `torch.nn.Linear`. Для $k = 2$ и $k = 3$ обучаются отдельные сети; первые две координаты трёхмерной сети не заменяют независимый двумерный эксперимент.

$$h = f\theta(z), \quad \hat{z} = g\varphi(h), \quad h \in R^k, \quad k \in \{2, 3\}$$

$$MSE = \sum_i \sum_j (z_{ij} - \hat{z}_{ij})^2 / (n \cdot 30)$$

Во внутренних скрытых слоях применяется ReLU. Узкий слой и выход декодировщика линейны: скрытые координаты и стандартизованные признаки могут быть отрицательными. Минимизируется средняя квадратичная ошибка восстановления MSE. Использован Adam с $\beta_1 = 0,9$, $\beta_2 = 0,999$ и $\varepsilon = 10^{-8}$, без weight decay.

Экспериментальный подбор параметров

Проверены две симметричные архитектуры: 30–16– k –16–30 и 30–32–16– k –16–32–30. Для каждой испытаны скорости обучения 0,001 и 0,003. Максимум — 1500 эпох; одна эпоха включает один шаг по всей обучающей части. Если валидационная MSE не уменьшается 150 эпох подряд, обучение завершается. Возвращаются веса эпохи с минимальной валидационной ошибкой.

k	Скрытые слои кодировщика	Скорость обучения	Лучшая эпоха	MSE обучения	MSE валидации
2	16	0,001	1428	0,262757	0,293421
2	16	0,003	538	0,270777	0,300369
2	32–16	0,001	402	0,293936	0,304206
2	32–16	0,003	222	0,288834	0,305639
3	16	0,001	486	0,220618	0,232095
3	16	0,003	223	0,214205	0,229429
3	32–16	0,001	393	0,198327	0,231716
3	32–16	0,003	238	0,184674	0,233804

Для $k = 2$ выбрана архитектура 30–16–2–16–30, скорость 0,001, эпоха 1428. Минимальная валидационная MSE составила 0,293421, тестовая — 0,287862.

Для $k = 3$ выбрана архитектура 30–16–3–16–30, скорость 0,003, эпоха 223. Минимальная валидационная MSE составила 0,229429, тестовая — 0,216510.

Сравнение относится к указанной сетке параметров и одной фиксированной инициализации. Оно не доказывает глобальную оптимальность выбранной сети. Обучение выполняется на CPU в float64 с одним потоком и включёнными детерминированными алгоритмами.

3 Обучение и скрытое представление

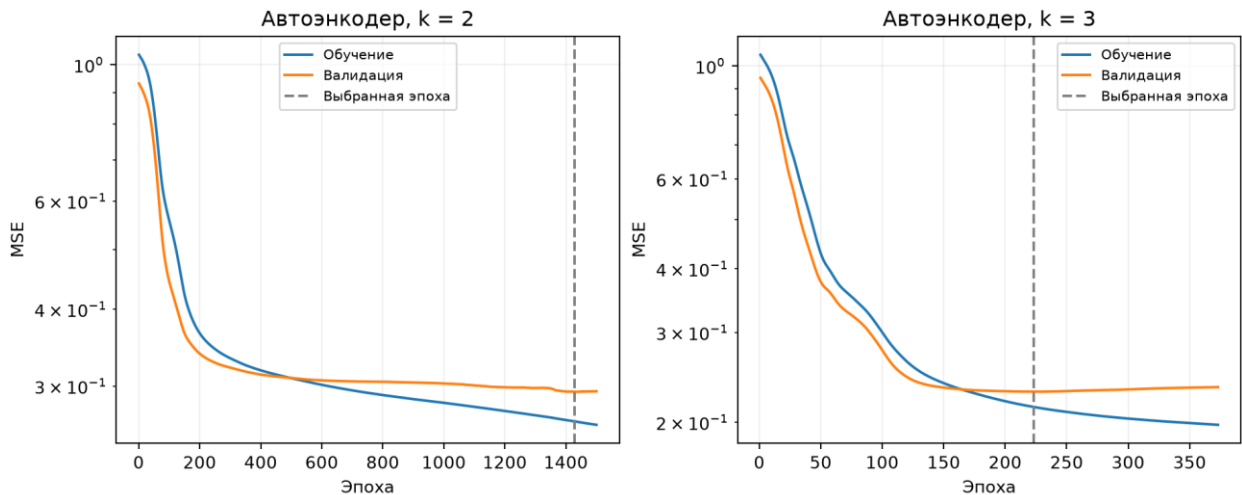


Рисунок 1 — Кривые обучения выбранных автоэнкодеров

Сплошные линии показывают ошибку на обучающей и валидационной частях; вертикальная пунктирная линия отмечает выбранную эпоху. Логарифмическая шкала позволяет сравнить начальный спад и поздние изменения. Кривые относятся к подбору параметров, а итоговые проекции ниже — к повторному обучению на полной выборке.

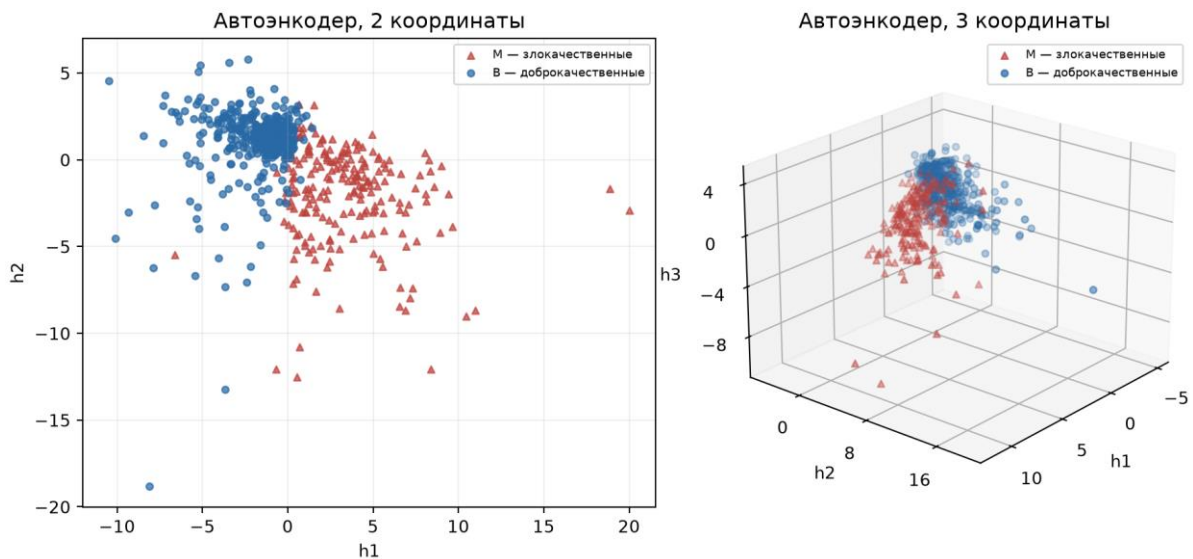


Рисунок 2 — Двумерное и трёхмерное представления автоэнкодера

Красные треугольники обозначают класс М, синие круги — класс В. Скрытое пространство отражает различия объектов, необходимые для восстановления признаков. В нём заметна структура классов, однако в пограничных областях сохраняется перекрытие. Обучение не использует диагноз и не оптимизирует точность классификации.

Оси $h1$, $h2$ и $h3$ — обученные скрытые координаты. Для нелинейного автоэнкодера они не обязаны быть ортогональными, упорядочены по дисперсии или совпадать с главными компонентами. Их масштаб и направление определяются весами сети.

4 Визуализация методом t-SNE

Метод t-SNE сопоставляет вероятностные соседства исходных объектов и их низкоразмерного представления, минимизируя KL-дивергенцию. В отличие от PCA он ориентирован прежде всего на локальную структуру. Использована `sklearn.manifold.TSNE` с `init="pca"`, `learning_rate="auto"`, `max_iter=1500`, методом Barnes–Hut и `random_state=42`.

Для каждой размерности проверены $\text{perplexity} = 20, 40$ и 60 . Перплексивность задаёт эффективный масштаб соседства и должна быть меньше числа объектов. Цветовые метки одинаковы во всех экспериментах; абсолютные размеры и расстояния между отдельными облаками t-SNE не трактуются как расстояния в исходных 30 признаках.

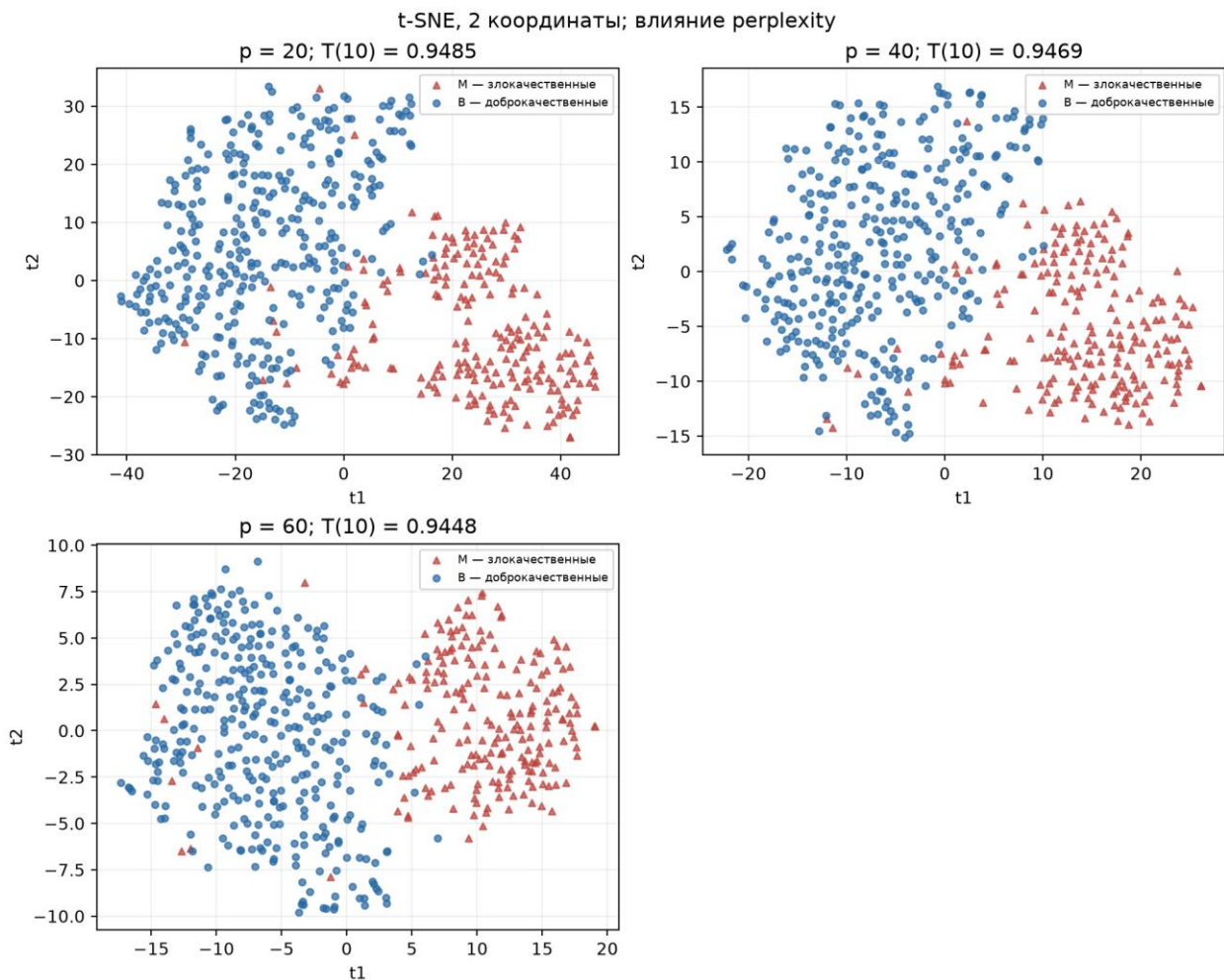


Рисунок 3 — Влияние perplexity на двумерную визуализацию t-SNE

Для двумерного представления наибольшая trustworthiness $T(10)$ получена при $\text{perplexity} = 20$: 0,948451. Этот вариант выбран по сохранению соседств без использования диагнозов. Изменение параметра меняет форму облаков, поэтому выбор не основан только на визуальном красивом разделении классов.

5 Трёхмерный t-SNE и оценка соседств

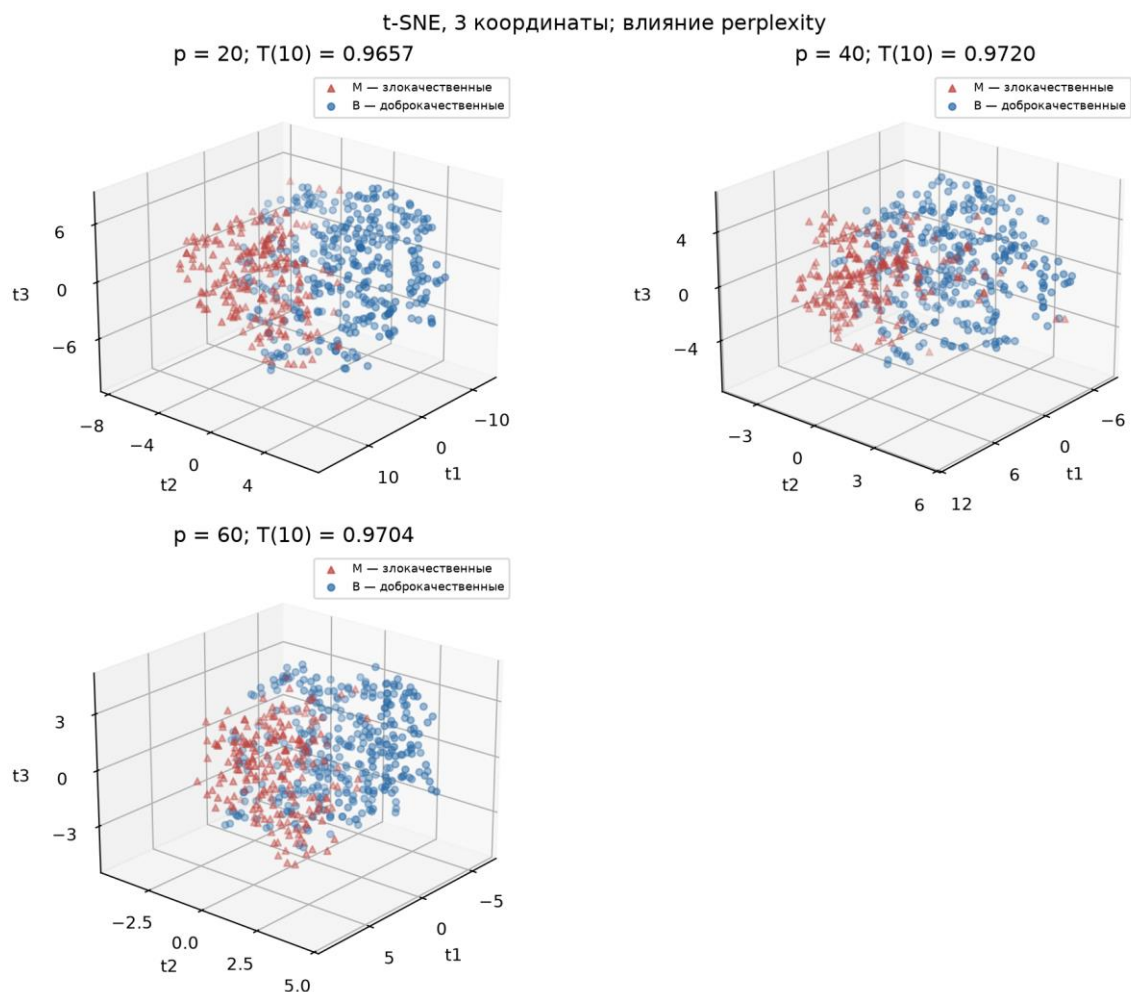


Рисунок 4 — Влияние perplexity на трёхмерную визуализацию t-SNE

Размерность	Perplexity	KL-дивергенция	T(10)
2	20	1,037734	0,948451
2	40	0,850243	0,946859
2	60	0,729892	0,944845
3	20	0,842602	0,965697
3	40	0,659314	0,971986
3	60	0,525900	0,970353

При трёх координатах выбран $\text{perplexity} = 40$, $T(10) = 0,971986$. Trustworthiness измеряет, насколько ближайшие соседи проекции соответствуют соседям в исходном стандартизованном пространстве. Использованы 10 соседей и евклидова метрика; более высокое значение лучше, максимум равен 1.

KL приведена как значение целевой функции каждого запуска. При разных perplexity меняются исходные вероятности, поэтому минимальная KL между такими запусками не является самостоятельным критерием выбора. У t-SNE нет используемого здесь декодировщика: MSE восстановления и доля объяснённой дисперсии для него не рассчитываются.

6 Применение метода главных компонент

Метод из первой лабораторной работы перенесён на 30 признаков WDBC. По стандартизированной матрице строится ковариация; `pumpu.linalg.eig` вычисляет собственные значения и векторы. Они сортируются по убыванию значений. Проверены вещественность результата, неотрицательность спектра и ортонормированность выбранного базиса.

$$S = Z^T Z / (n - 1); \quad T_k = Z V_k; \quad Z_k = T_k V_k^T$$

$$R_k = 100 \cdot \sum_{j=1}^k \lambda_j / \sum_{j=1}^{30} \lambda_j$$

Результат независимо сопоставлен с `sklearn.decomposition.PCA` при `svd_solver="full"`. Поскольку знак собственного вектора произволен, для проверки подпространства сравниваются проекторы $V_k V_k^T$. Для графиков максимальный по модулю коэффициент каждого вектора делается положительным.

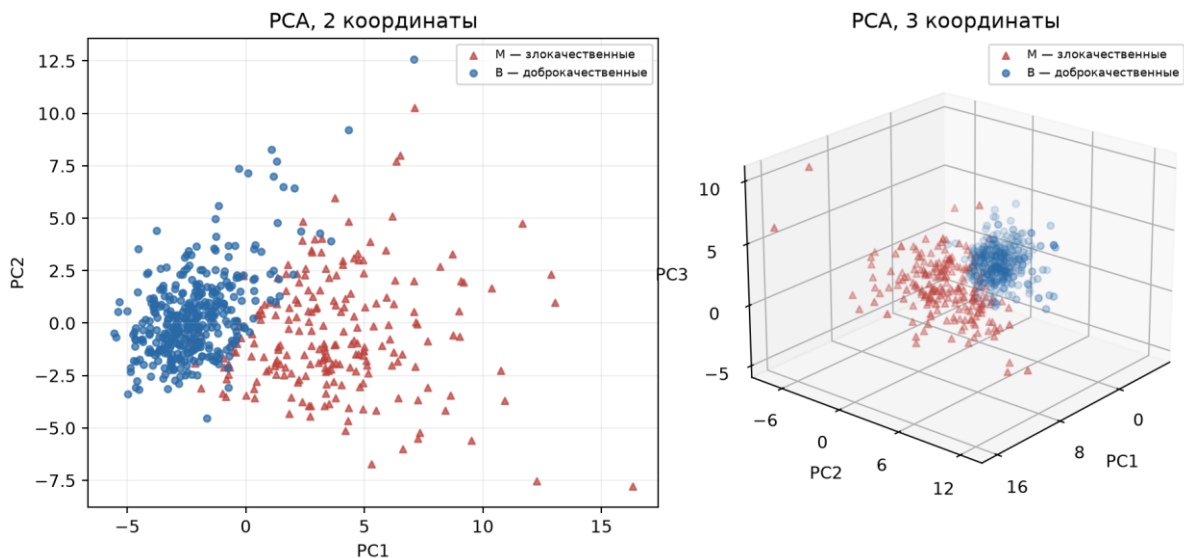


Рисунок 5 — Проекция WDBC на две и три главные компоненты

k	Сохранено, %	Потеряно, %	MSE полной выборки
2	63,2432	36,7568	0,366922
3	72,6364	27,3636	0,273155

Две компоненты сохраняют 63,24% стандартизированной дисперсии, три — 72,64%. Третья компонента добавляет 9,39 процентного пункта. Различия классов частично проявляются уже на плоскости, но облака перекрываются.

Проверено равенство ошибки восстановления сумме отброшенных собственных значений с множителем $(n - 1)/(n \cdot 30)$. Доля сохранённой дисперсии относится к стандартизованным признакам и не является долей правильно распознанных диагнозов.

7 Сравнение методов

Метод	k	MSE полной выборки	MSE теста	T(10) полной выборки
PCA	2	0,366922	0,360687	0,871348
Автоэнкодер	2	0,267602	0,287862	0,908507
t-SNE p=20	2	—	—	0,948451
PCA	3	0,273155	0,280674	0,929201
Автоэнкодер	3	0,220465	0,216510	0,946183
t-SNE p=40	3	—	—	0,971986

MSE полной выборки относится к моделям, обученным на всех объектах после полной стандартизации. MSE теста относится к моделям, обученным только на 341 объекте, с масштабами этой обучающей части. Поэтому сравниваются методы внутри каждого столбца; различие полной и тестовой MSE не следует целиком приписывать переобучению.

При $k = 2$ тестовая ошибка автоэнкодера равна 0,287862, PCA — 0,360687. В данном разбиении ошибка автоэнкодера на 20,19% меньше. Нелинейная модель может учитывать зависимости, недоступные линейному подпространству, однако результат зависит от архитектуры и обучения.

При $k = 3$ тестовая ошибка автоэнкодера равна 0,216510, PCA — 0,280674. В данном разбиении ошибка автоэнкодера на 22,86% меньше. Нелинейная модель может учитывать зависимости, недоступные линейному подпространству, однако результат зависит от архитектуры и обучения.

Интерпретация различий

PCA даёт упорядоченные ортогональные направления и точную оценку потерянной дисперсии. Его удобно использовать как быстрый воспроизводимый базовый метод. Оптимальность PCA по квадратичной ошибке относится к линейным ортогональным проекциям и не исключает меньшей ошибки нелинейного автоэнкодера.

Автоэнкодер предоставляет кодировщик и декодировщик: новые объекты можно стандартизовать сохранёнными параметрами и преобразовать обученной сетью. За эту гибкость приходится платить подбором параметров. Ошибка восстановления и сохранение соседств оценивают разные свойства, поэтому лучшая MSE не гарантирует лучшую визуализацию.

t-SNE удобен для исследования локальных групп и имеет высокие значения T(10) в выполненных запусках. Однако оси не обладают смыслом главных компонент; метод sklearn TSNE не предоставляет transform для новых объектов. Качество рисунка на всей выборке не является оценкой работы на независимых новых данных.

Тестовая часть не участвовала в выборе автоэнкодера. Оценка выполнена на одном фиксированном разбиении и не сопровождается доверительными интервалами. Ни один эксперимент не обучает классификатор, поэтому на основании этих графиков не делается вывод о точности диагностики.

8 Программа и проверка корректности

Данные WDBC входят в scikit-learn: после установки библиотек не требуются отдельный CSV, архив или доступ к сети. Основной запуск обучает восемь вариантов сетей, выбирает два автоэнкодера, выполняет шесть запусков t-SNE и вычисляет PCA для двух размерностей.

Команды запуска

```
python -m pip install numpy "scikit-learn>=1.5" matplotlib torch
python iad_lab2_variant1.py
python iad_lab2_variant1.py --no-show
python iad_lab2_variant1.py --self-test
```

Обычный запуск открывает графики matplotlib; трёхмерные рисунки можно вращать мышью. Флаг --no-show сохраняет результаты без окон. Каталог results_lab2 создаётся рядом со скриптом; другой каталог задаётся параметром --out. Параметр --epochs изменяет предел обучения; для воспроизведения отчёта используется значение по умолчанию 1500.

В каталог результатов записываются пять PNG, JSON с параметрами и метриками, CSV с координатами и кривыми обучения, а также два файла весов .pt с архитектурой и параметрами стандартизации. Исходный код для вычислений остаётся одним Python-файлом.

Проверки реализации

Режим --self-test проверяет непересечение обучающей, валидационной и тестовой частей; нулевые средние и единичные выборочные дисперсии; совпадение проекторов ручного PCA и sklearn для $k = 2, 3$ и 30; формулу ошибки реконструкции; уменьшение MSE при обучении; размеры выходов и повторяемость автоэнкодера при одинаковом seed. Эти проверки пройдены.

Основной запуск дополнительно проверяет размер 569×30 , численность классов, конечность данных и скрытых координат, совпадение спектров PCA и отсутствие расходимости обучения. Таблицы и изображения отчёта получены этим запуском, а не заданы вручную.

9 Выводы

Для варианта 1 выполнено снижение размерности WDBC с 30 признаков до двух и трёх координат тремя способами. Построены отдельные визуализации автоэнкодера, t-SNE и PCA; классы обозначены едиными цветами и формами маркеров.

Архитектура и скорость обучения автоэнкодера выбраны экспериментально по валидационной ошибке среди четырёх комбинаций для каждой размерности. Сохранены веса лучшей эпохи; тестовая часть использована только после выбора. Нелинейное представление обеспечивает восстановление признаков и выявляет структуру классов при сохранении пограничного перекрытия.

Для t-SNE проверены perplexity 20, 40 и 60. По T(10) выбраны значения 20 для двумерного и 40 для трёхмерного представления. Сравнение показало зависимость геометрии рисунка от параметра соседства. Расстояния между облаками и размеры кластеров не интерпретируются как точное воспроизведение исходной геометрии.

PCA сохраняет 63,24% дисперсии при двух компонентах и 72,64% при трёх. Ручное вычисление через eig согласуется с библиотечной реализацией. Увеличение размерности улучшает восстановление PCA, однако даже три компоненты не сохраняют всю дисперсию 30 признаков.

Выбор метода зависит от задачи: PCA удобен для линейного сжатия с интерпретируемыми потерями, автоэнкодер — для нелинейного кодирования с восстановлением, t-SNE — для локальной визуализации. Качество восстановления, сохранение соседств и визуальное разделение классов не являются взаимозаменяемыми критериями.